

CHAPTER 02

Software Architecture

ACRONYMS

AD	Architecture Description
ADL	Architecture Description Language
API	Application Programming Interface
ASR	Architecturally Significant Requirement
ATAM	Architectural Tradeoff Analysis Method
IDL	Interface Description Language
MVC	Model View Controller
QAW	Quality Attribute Workshop
RA	Reference Architecture
REST	Representational State Transfer
SAAM	Software Architecture Analysis Method
UML	Unified Modeling Language

INTRODUCTION

This chapter considers software architecture from several perspectives: concepts; representation and work products; context, process and methods; and analysis and evaluation.

In contrast to the previous edition, this edition creates a software architecture knowledge area (KA), separate from the Software Design KA, because of the significant interest and growth of the discipline since the 1990s.

BREAKDOWN OF TOPICS FOR SOFTWARE ARCHITECTURE

The breakdown of topics for the Software Architecture KA is shown in Figure 2.1.

1. Software Architecture Fundamentals
- [2*,c1][29*,appendix C][38*,c2][41*,c1-3]
- 1.1. The Senses of “Architecture” [2*,c2]
- [6*,c6.1][29*,c6]

Software engineering and related disciplines use many senses of “architecture.” First, “architecture” often refers to a discipline: the art and science of constructing things — in this case, software-intensive systems. The discipline involves concepts, principles, processes and methods the community has discovered and adopted.

Second, “architecture” refers to the various processes through which that discipline is realized. Software architecture is also considered part of Software Design; generally considered a multistage process, divided into the following stages:

- Architectural design stage
- High-level design stage
- Detailed design stage

Software design is the focus of Chapter 3. This chapter focuses on *architecting* and *architectural design*.

Third, “architecture” refers to the *outcome* of applying architectural design discipline and processes to devise architectures for software systems. Architectures as outcomes are expressed in *architecture descriptions*. This is discussed in topic *Software Architecture Description*. The concept of architecture has evolved, and many definitions are in use today. One early definition of architecture, from 1990, emphasized software structure:

Architecture. The organizational structure of a system or component. [from: IEEE

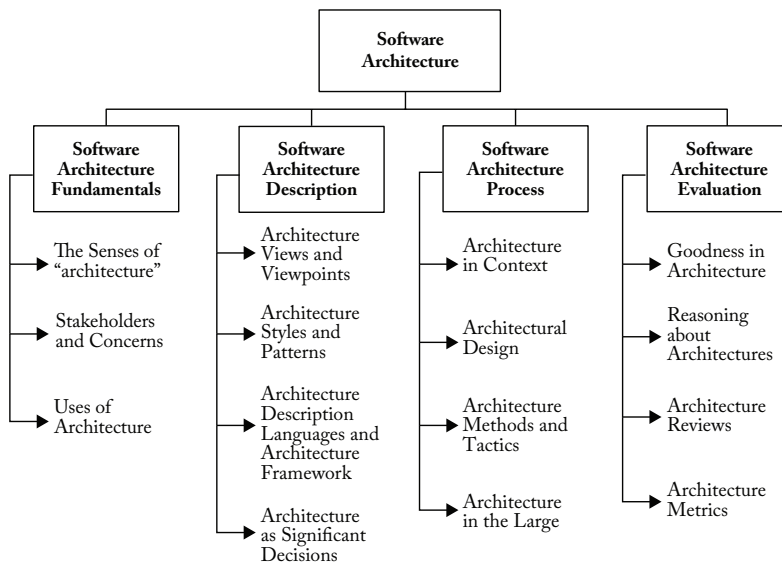


Figure 2.1. Breakdown of Topics for the Software Architecture KA

Std 610.12–1990, *IEEE Glossary of Software Engineering Terminology*]

This definition did not do justice to evolving thinking about architecture; e.g., this definition does not allow us to distinguish the detailed design of a module from its Makefile. Either example reflects an *organizational structure* of the software system or component but should not be considered architecture. Moreover, emphasis on the structure was often limited to the code's structure and failed to encompass all the structures of the software system:

The software architecture of a system is the set of structures needed to reason about the system. These structures comprise software elements, relations among them, and properties of both. [2]*

During the mid-1990s, however, software architecture emerged as a broader discipline involving a more generic study of software structures and architectures. Many software system structures are not directly reflected in the code structure. Both types of structure have implications for the system as a whole: What behaviors is the system capable

of? What interactions does it have with other systems? How are properties like safety and security handled? The recognition that software contains many different structures has prompted discussion of a number of interesting concepts about software architecture (and software design more generally) leading to current definitions such as:

architecture (*of a system*). *fundamental concepts or properties of a system in its environment embodied in its elements, relationships, and in the principles of its design and evolution [23]*

Key ideas in that definition are the following: (1) Architecture is about what is *fundamental* to a software system; not every element, interconnection, or interface is considered fundamental. (2) Architecture considers a system *in its environment*. Much like building architecture, software architecture is outward-looking; it considers a system's context beyond its boundaries including the people, organizations, software, hardware and other devices with which the system must interact.

1.2. Stakeholders and Concerns

[2*, c3-14] [38*, c8-9] [41*, c3]

A software system has many *stakeholders* with varying roles and interests relative to that system. These varying interests are termed *concerns*, following Dijkstra's *separation of concerns*:

Let me try to explain to you, what to my taste is characteristic for all intelligent thinking. It is, that one is willing to study in depth an aspect of one's subject matter in isolation for the sake of its own consistency, all the time knowing that one is occupying oneself only with one of the aspects. We know that a program must be correct and we can study it from that viewpoint only; we also know that it should be efficient and we can study its efficiency on another day, so to speak. In another mood we may ask ourselves whether, and if so: why, the program is desirable. But nothing is gained — on the contrary! — by tackling these various aspects simultaneously. It is what I sometimes have called "the separation of concerns", which, even if not perfectly possible, is yet the only available technique for effective ordering of one's thoughts, that I know of. This is what I mean by "[focusing] one's attention upon some aspect": it does not mean ignoring the other aspects, it is just doing justice to the fact that from this aspect's point of view, the other is irrelevant. It is being one- and multiple-track-minded simultaneously. [12]

What is fundamental about a system varies according to stakeholders' concerns and roles. The software structures, therefore, also vary with stakeholder roles and concerns. (See also topic *Design Methods* in Software Design KA.)

A software system's customer is most interested in when the system will be ready and how much it will cost to build and operate. Users are most interested in what it does and how to use it. Designers and programmers building the system have their own concerns, such as whether an algorithm will meet the system requirements. Those responsible for ensuring the system is safe to operate have different concerns.

affordability, agility, assurance, autonomy, availability, behavior, business goals and strategies, complexity, compliance with regulation, concurrency, control, cost, data accessibility, deployability, disposability, energy efficiency, evolvability, extensibility, feasibility, flexibility, functionality, information assurance, inter-process communication, interoperability, known limitations, maintainability, modifiability, modularity, openness, performance, privacy, quality of service, reliability, resource utilization, reusability, safety, scalability, schedule, security, system modes, software structure, subsystem integration, sustainability, system features, testability, usability, usage, user experience

Figure 2.2. Examples of Architectural Concerns

Concerns encompass a broad range of issues, possibly pertaining to any influence on a system in its environment, including developmental, technological, business, operational, organizational, political, economic, legal, regulatory, ecological and social influences. Like software requirements, they may be classified as functional, non-functional or constraint. (See Software Requirements KA.) Concerns manifest in various familiar forms, including requirements, quality attributes or "ilities", emergent properties (which may be either desired or prohibited) and various kinds of constraints (as listed above). See Software Quality KA. Topic 2, *Software Architecture Description*, shows how concerns shape architecture and the work products describing those architectures. Examples of concerns are depicted in Figure 2.2. Concerns are not static; concerns evolve over the life cycle of a system and as technologies, policies and other influences evolve. For example, due to increased awareness of climate change, there is growing interest in concerns such as energy efficiency, and sustainability [24].

1.3. *Uses of Architecture*

[2*, c24] [38*, c30]

A principal use of a software system's architecture is to give those working with it a shared understanding of the system to guide its design and construction. An architecture also serves as a preliminary conception of the software system that provides a basis to analyze and evaluate alternatives. A third common usage is to enable reverse engineering (or *reverse architecting*) by helping those working with it to understand an existing software system before undertaking maintenance, enhancement or modification. To support these uses, the architecture should be documented (see topic *Software Architecture Description*).

Conway's Law posits that "organizations which design systems ... are constrained to produce designs which are copies of the communication structures of these organizations" [11]. Empirical studies have observed that the architectures of these systems often mirror the communications structures of those organizations [28]. Depending on the software system and the organization, this can be a strength or a weakness. The architecture can enhance communication within a large team or compromise it. Each part of the organization can base its planning, costing and scheduling activities upon its knowledge of the architecture. Creating a well-planned and documented architecture is one approach to increasing the applicability and reusability of software designs and components. The architecture forms the basis for design families of programs or software product lines. This can be done by identifying commonalities among members of such families and by designing reusable and customizable components to account for the variability among family members.

2. **Software Architecture Description**

[2*, c1.2, 22] [38*, c12-13] [40*, c6]
[41*, c6-7]

In topic 1, *Software Architecture Fundamentals*, a software architecture was defined as the

fundamental concepts or properties of a software system in its environment. But each stakeholder can have a different notion of what is fundamental to that software system, given their perspective. Having a mental model of a system's architecture is perhaps fine for small systems and for individuals working alone. However, for large, complex systems developed and operated by teams, a tangible representation is invaluable, especially as the conception of the system evolves, and as people join or leave the team. Having a concrete representation as a work product can also serve as a basis to analyze the architecture, organize its design and guide its implementation. These work products are called *architecture descriptions* (ADs).

ADs document an architecture for a software system. It is targeted to those stakeholders of the system who have concerns about the software system which are answered by the architecture. As noted in topic 1, *Software Architecture Fundamentals*, a primary audience comprises the designers, engineers and programmers whose concerns pertain to constructing the system. For these stakeholders, ADs serve as a *blueprint* to guide the construction of the software system. For others, the AD is a basis for their work — for example, testing and quality assurance, certification, deployment, operation, and maintenance and future evolution.

Historically, ADs used text and informal diagrams to convey the architecture. However, the diversity of stakeholder audiences and their different concerns have led to a diversity of representations of the architecture. Notations should be chosen based on the need, purpose and the utility of those choices (such as understandability, familiarity) for the stakeholders who need that information. Often, these representations are specialized based upon existing practices of the communities or disciplines involved to effectively address this variety of stakeholders and concerns (see Software Design KA and Software Engineering Models and Methods KA). These various representations are called *architecture views*.

2.1. *Architecture Views and Viewpoints*

[6*, c7-9] [29*, c8] [38*, c3] [40*, c6.2]

An *architecture view* represents one or more aspects of an architecture to address one or more concerns [38*]. Views address distinct concerns — for example, a logical view (depicts how the system will satisfy the functional requirements); a process view (depicts how the system will use concurrency); a physical view (depicts how the system is to be deployed and distributed) and a development view (depicts how the top-level design is broken down into implementation units, the dependencies among those units and how the implementation is to be constructed). Separating concerns by view allows interested stakeholders to focus on a few things at a time and offers a means of managing the architecture’s understandability and overall complexity.

Architecture practice has evolved from the use of text and informal diagrams to the use of more rigorous representations. Each architecture view depicts architectural elements of the system using well-defined conventions, notations and models [38*]. The conventions for each view are documented as an *architecture viewpoint* [23]. Viewpoints guide the creation, interpretation and uses of architecture views. Each viewpoint links stakeholder audience concerns with a set of conventions. In model-based architecting, each view can be machine-checked against its viewpoint.

Common viewpoints include the module viewpoint, used to express a software system’s implementation in terms of its modules and their organization [2*]; the component and connector viewpoint, used to express the software’s large-scale runtime organization and interactions [2*]; the logical viewpoint, used to express fundamental concepts of the software’s domain and capability [25]; the scenarios/use cases viewpoint, used to express how users interact with the system [25]; the information viewpoint, used to express a system’s key information elements and how they are accessed and stored [38*]; and the deployment viewpoint, used to express how a system is configured and deployed for operation [38*].

Other documented viewpoints include viewpoints for availability, behavior, communications, exception handling, performance, reliability, safety and security.

Each viewpoint provides a vocabulary or language for talking about a set of concerns and the mechanisms for addressing them. The viewpoint language gives stakeholders a shared means of expression. Viewpoints need not be limited to one software system but are reusable by an organization or application community for many similar systems. When generic representations such as Unified Modeling Language (UML) are used, they can be specialized to the system, its domain or the organizations involved. (See section 2.3 *Architecture Description Languages and Architecture Frameworks*.)

Beyond specifying forms of representation, an architecture viewpoint can capture the ways of working within a discipline or community of practice. For example, a software reliability viewpoint captures existing practices from the software reliability community for identifying and analyzing reliability issues, formulating alternatives and synthesizing and representing solutions. Like engineering handbooks, general-purpose and specialized viewpoints provide a means to document repeatable or reusable approaches to recurring software issues. Clements *et al.* have introduced viewtypes which establish a 3-way categorization of viewpoints. These categories are module, component and connector, and allocation viewtypes [9].

Architecture descriptions frequently use *multiple* architecture views to represent the diverse structures needed to address different stakeholders’ various concerns. There are two common approaches to the construction of views: the *synthetic approach* and the *projective approach*. In the synthetic approach, architects construct views of the system-of-interest and integrate these views within an architecture description using correspondence rules. In the projective approach, an architect derives each view through some routine, possibly mechanical, procedure of extraction from a single unified model (or “uber model”) [23].

A consequence of introducing multiple views into an AD is a potential mismatch between the views. Are they consistent? Are they describing the same system? This has been called the *multiple views problem* [39]. The projective approach limits possible inconsistencies, since views are derived from a single (presumably consistent) model, but at the cost of expressiveness: the underlying model may not be capable of capturing arbitrary concerns. Under the synthetic approach, architects integrate views into a whole, using linkages or other forms of traceability to cross-reference view elements to achieve consistency [23,25]. Viewpoints often include rules for establishing consistency or other relationships among views.

2.2. *Architecture Patterns, Styles and Reference Architectures* [2*, c2.12] [6*, c6,15] [38*, c11] [40*, c6.3] [41*, c11]

Inspired by its use in the long history of the architecture of buildings, an *architectural style* is a particular manner of construction yielding a software system's characteristic features. An architectural style often expresses a software system's large-scale organization. In contrast, an *architectural pattern* expresses a common solution to a recurring problem *within* the context of a software system — it need not apply to the whole system. Design patterns are discussed in section 4.4 of Software Design KA.

Various architectural styles and patterns have been documented [7,39]:

- Adaptive systems (e.g., microkernel, reflection and meta-level architectures)
 - Virtual machines (e.g., interpreters, rule-based, process control)
- Pattern catalogs* (or *systems of patterns*) are used to express architectural styles and solutions through coordinated sets of patterns. Examples of pattern catalogs are [7], [19] for n-tier architectures, [13] for service-oriented architecture and [37] for microservice architectures. Pattern catalogs are not limited to architecture styles and can be focused on addressing specific concerns, such as security [17].
- There is no strict dividing line between architectural styles and patterns. Both patterns and styles provide solutions to specific problems in given contexts. An architectural style expresses the global aspects of a system or subsystem by defining its major parts of that (sub)system and how they interact [7,38*]. An architectural style can be expressed as an architectural pattern [7]. Architectural patterns exist at varying scales and could apply once to a single element of a system or be applied repeatedly throughout a system.
- In relation to architecture viewpoints, which provide the languages for talking about various aspects of software systems, a unifying notion is that both patterns and styles are *idioms* in those languages for expressing particular aspects of architectures (and designs, see section 4.4 *Design Patterns* in Software Design KA). An architectural pattern or style uses a vocabulary, drawn from the viewpoint's language, in a specified way, to talk about view elements, including element and relation types and their instances, and constraints on combining them [23,39]. In this way, viewpoints, patterns and styles are mechanisms for codifying recommended practices to facilitate reuse.
- A *reference architecture* (RA) is an architecture constraining or guiding other architectures. Documented as a reference architecture description, an RA provides a common basis for the development of architectures for individual systems, product lines or families of systems and application domains. Reference architectures capture commonalities to

promote ease of development, integration and interoperability and other kinds of standardization. Reference architectures have been developed and used in many domains including automotive systems, healthcare, Internet of Things, cloud computing, avionics, manufacturing and telecommunications.

2.3. *Architecture Description Languages and Architecture Frameworks* [2*, c22] [29*, c11] [38*, app] [41*, c6-7]

An architecture description language (ADL) is a domain-specific language for expressing software architectures. ADLs arose from module interconnection languages [36] for programming in the large. Some ADLs target a single application domain or architectural style (such as MetaH for avionics systems in an event-driven style), others are wide spectrum to frame concerns across the enterprise (such as ArchiMate™). UML has frequently been used as an ADL due to its widespread use in software design activities [41*]. ADLs often provide capabilities beyond description to enable architecture analysis or code generation.

An *architecture framework* captures the “conventions, principles and practices for the description of architectures established within a specific domain of application and/or community of stakeholders” [23]. Frameworks codify recommended practices within a specific domain and are implemented as an interlocking set of viewpoints or ADLs. Examples are AUTOSAR for the automotive industry, OMG’s Unified Architecture Framework (UAF®) and ISO Reference Model for Open Distributed Processing.

2.4. *Architecture as Significant Decisions* [38*, c8] [40*, c6.1]

Architectural design is a creative process. During this activity, architects make many decisions that profoundly affect the architecture, the downstream development process and the software system. Many factors affect decision-making, including prominent concerns of stakeholders for the

software system, its requirements, and the available resources during development and throughout the life cycle. The impact on quality attributes and trade-offs among competing quality attributes are often the basis for design decisions.

The architectural design activity creates a network of decisions as its outcome, with some decisions deriving from prior decisions. Decisions can be explicitly documented, along with an explanation of the rationale for each nontrivial decision. Decision analysis provides one approach to architecture evaluation. (See topic 4, Software Architecture Evaluation.)

Architecture rationale captures *why* an architectural decision was made. This includes assumptions made before the decision, alternatives considered, and trade-offs or criteria used to select an approach and reject others. Recording rejected decisions and the reasons for their rejection can also be useful. In the future, this could either prevent a software project from making a poor decision — one rejected earlier for forgotten reasons — or allow the development to recognize that relevant conditions have changed and that they can revisit the decision.

Architectural technical debt has been introduced to reflect that today’s decisions for an architecture may have significant consequences later in the software system’s life cycle. Decisions deferred can compromise its maintainability or the future evolvability, and that debt will have to be paid — typically by others, not necessarily by those who caused the debt. Such debt has an economic impact on the system’s future development and operations. For example, when a software project has limited time, it may develop an initial design with little concern for modularity for its first release. The lack of modularity can adversely affect the development time for subsequent releases, impact developers, and perhaps compromise future maintainability of the system. Additional functionality can be added later only by doing extensive refactoring which impacts future timelines and introduces additional defects. [26]. Architectural

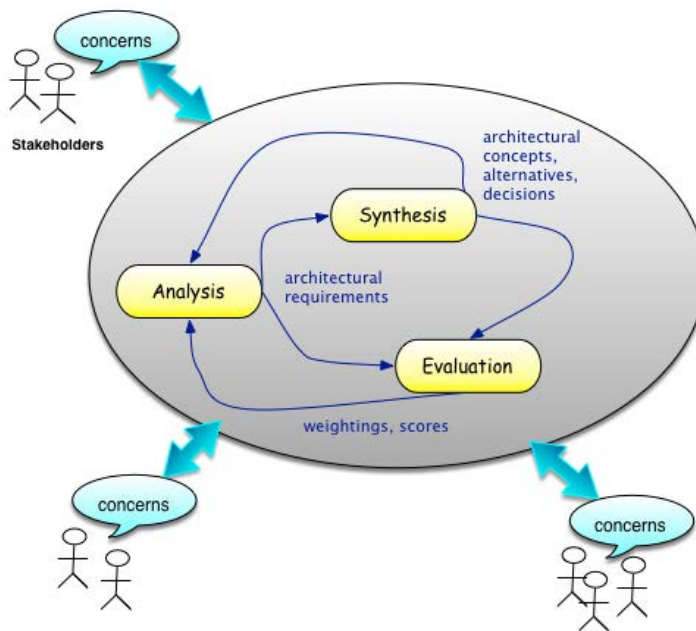


Figure 2.3. A general model of architectural design

technical debt can be analyzed and managed, like other concerns, using models and viewpoints [27].

3. Software Architecture Process

[29*, c9] [38*, c6-7] [41*, c4]

This section outlines a general model of an architectural design process. It is used to demonstrate how architectural design fits into the general context of software engineering processes (see Software Engineering Process KA) and as a framework for understanding the many architecture methods currently in use. It also recognizes that architectural design can take place in a variety of contexts.

3.1. Architecture in Context

[29*, c12-13] [41*, c2]

Architecture occurs in several contexts. In the traditional life cycle, there is an architectural design stage driven by software system requirements (see Software Requirements KA). Some requirements will be *architectural drivers*, influencing major decisions about the

architecture, while other requirements are deferred to subsequent stages of the software process, such as design or construction.

In product line or product family settings, a product line/family architecture is developed against a basic set of needs, requirements and other factors. That architecture will be the starting point for one or more product instances developed against specific product requirements, building upon the product baseline.

In agile approaches, there is not usually an architecture design stage. The only architecture description might be the code itself. In some agile practices, the software architecture is said to “emerge” from coding the system based on user stories through a rapid series of development cycles. Although this approach has had some success with user-centric information systems, it is difficult to ensure an adequate architecture *emerges* for other classes of applications, such as embedded and cyber-physical systems, when critical architectural properties might not be articulated by any user stories.

In enterprise and system-of-systems contexts, as in product lines and families, the overarching architecture (of the enterprise,

system or product line/family) provides primary requirements and guidance on the form and constraints upon the software architecture. This baseline can be enforced through specifications, additional requirements, application programming interfaces (APIs) or conformance suites.

3.1.1. *Relation of Architecture to Design* [40*, c6] [41*, c2]

Design and architecture are often blurred. It has been said that architecture is the set of decisions that one cannot trust to designers. In fact, architecture emerged out of software design as the discipline matured, largely since the 1990s. There are various contrasts: design often focuses on an established set of requirements, whereas architecture often must shape the requirements through negotiation with stakeholders and requirements analysis. In addition, architecture often must recognize and address a wider range of concerns that may or may not end up as requirements on the software system of interest.

3.2. *Architectural Design* [2*, c19–23]

Architectural design is the application of design principles and methods within a process to create and document a software architecture. There are many architecture methods for carrying out this activity. This section describes a general model of architectural design underlying various architecture methods based upon [20].

Architectural design involves identifying a system's major components; their responsibilities, properties, and interfaces; and the relationships and interactions among them and with the environment. In architectural design, fundamentals of the system are decided, but other aspects, such as the internal details of major components are deferred.

Typical concerns in architectural design include the following:

- Overall architecture styles and computing paradigms

- Large-scale refinement of the system into key components
- Communication and interaction among components
- Allocation of concerns and design responsibilities to components
- Component interfaces
- Understanding and analysis of scaling and performance properties, resource consumption properties, and reliability properties
- Large-scale/system-wide approaches to dominating concerns (such as safety and security, where applicable)

An overview of architectural design is presented in Figure 2.3.

Architectural design is iterative, comprising three major activities: analysis, synthesis and evaluation. Often, all three major activities are performed concurrently at various levels of granularity.

3.2.1. *Architecture Analysis* [2*, c19] [41, c8]

Architecture analysis gathers and formulates *architecturally significant requirements* (ASRs), defined as any “requirement upon a software system which influences its architecture” [31]. Architecture analysis is based on identified concerns and on understanding the software's context, including known requirements, stakeholder needs and the environment's constraints. ASRs reflect the design problems the architecture must solve. Often the combination of initial requirements and known constraints cannot be satisfied without consequences to cost, schedule, etc. In such cases, negotiation is used to modify incoming needs, requirements and expectations to make solutions possible. Architecture analysis produces ASRs, initial system-wide decisions and any overarching system principles derived from the context (see *Architecture in Context*).

3.2.2. *Architecture Synthesis* [2*, c20]

Architecture synthesis develops candidate solutions in response to the outcomes of

architecture analysis. Synthesis proceeds by working out detailed solutions to design problems identified by ASRs, and makes trade-offs to accommodate interactions between those solutions. These outcomes feed back to architecture analysis resulting in elaborated ASRs, principles and decisions which then lead to further detailed solution elements.

3.2.3. *Architecture Evaluation* [2*, c21] [38*, c14]

Architecture evaluation validates whether the chosen solutions satisfy ASRs and when and where rework is needed. Architecture evaluation methods are discussed in topic 4 *Software Architecture Evaluation*.

3.3. *Architecture Practices, Methods, and Tactics* [2*, c3.4] [29*, c10] [38*, c9-14]

There are a number of documented architecture methods (see Further Readings for a list).

3.4. *Architecting in the Large* [29*, c12, 14] [40*, c19]

Architectural design denotes a specific stage of the life cycle, but is only one part of software architecting. Software architecting does not occur in a vacuum, as noted in section 3.1 *Architecture in Context*, but in an environment that often includes other architectures. For example, an application architecture should conform to an enterprise architecture; to “play well” in a system of systems, the architecture of each constituent system should conform to the system of systems architecture. In such cases, these relations need to be reflected as ASRs on the software being architected. Many software architecting activities and principles are not limited to software but equally apply to systems and enterprise architecting [29]. Weinreich and Buchgeher have extended Hofmeister et al.’s model used in section 3.2 *Architectural Design* to include these activities [42]:

- *architecture implementation*: overseeing implementation and certifying

that implementations conform to the architecture

- *architecture maintenance*: managing and extending the architecture following its implementation
- *architecture management*: managing an organization’s portfolio of interrelated architectures
- *architecture knowledge management*: extracting, maintaining, sharing and exploiting reusable architecture assets, including decisions, lessons learned, specifications and documentation across the organization

4. *Software Architecture Evaluation* [2*, c21] [38*, c14] [41*, c8]

4.1. *“Goodness” in Architecture* [2*, c1.3, 2] [6*, c17]

Architecture analysis takes place throughout the process of creating and sustaining an architecture. *Architecture evaluation* is typically undertaken by third parties at determined milestones as a form of assessment.

Given the multi-concern, multi-disciplinary nature of software architecture, there are many aspects to what makes an architecture “good.” The Roman architect Vitruvius posited that all buildings should have the attributes of *firmitas*, *utilitas* and *venustas* (translated from Latin as strength, utility and beauty).

Of a software system and its architecture, one can ask:

- Is it robust over its lifetime and possible evolution?
- Is it fit for its intended use?
- Is it feasible and cost-effective to construct software systems using this architecture?
- Is it, if not beautiful, then at least clear and understandable to those who must construct, use and maintain the software?

Each architecture concern may be a basis for evaluation. Evaluation is conducted against requirements (when available) or against need, expectations and norms (in other situations).

A “good” architecture should address not only the distinct concerns of its stakeholders, but also the consequences of their interactions. For example, a secure architecture may be excessively costly to build and verify; an easy-to-build architecture may not be maintainable over the system’s lifetime if it cannot incorporate new technologies.

The Architecture Tradeoff Analysis Method (ATAM) [10] provides a methodical approach to evaluating software architectures based on quality attributes in a utility tree and scenarios illustrating the qualities. Analysis of tradeoffs among competing quality requirements and their architectural approaches are the key to the architecture evaluation. Clements, et al. describe several methods for evaluation including ATAM, Software Architecture Analysis Method (SAAM), and Quality Attribute Workshops (QAW) [10]. The SARA Report defines a general framework for software architecture evaluation [31].

4.2. Reasoning about Architectures

[38*, c10]

Each architecture concern has a distinct basis for evaluation. Evaluation is most effective when it is based upon robust, existing architecture descriptions. ADs can be queried, examined and analyzed. For example, evaluation of functionality or behavior benefits from having an explicit architecture view or other representation of that aspect of the system to study. Specialized concerns such as reliability, safety and security often rely on specialized representations from the respective discipline.

Often architecture documentation is unfinished, incomplete, out of date or nonexistent. In such cases, the evaluation effort must rely on the knowledge of participants as a primary information source.

Use cases are frequently used to check an architecture’s completeness and consistency (see Software Engineering Models and Methods KA) by comparing the steps in the

use case to the software architecture elements that would be involved in carrying out those steps [23].

For a general framework for reasoning about various concerns, see Bass et al. [3].

4.3. Architecture Reviews

[2*, c21]

Architecture reviews are an effective approach to assess an architecture’s status and quality and identify risks by assessing one or more architecture concerns [1]. Many reviews are informal or expertise-based, and some are more structured, organized around a checklist of topics to cover. Parnas and Weiss proposed an effective approach to conducting reviews, called *active reviews* [33], where instead of checklists, each evaluation item entails a specific activity by a reviewer to obtain the needed information.

Many organizations have institutionalized architecture review practices. For example, an industry group developed a framework for defining, conducting and documenting architecture reviews and their outcomes [31].

4.4. Architecture Metrics

[2*, c23]

An *architecture metric* is a quantitative measure of a characteristic of an architecture. Various architecture metrics have been defined. Many of these originated as design or code metrics that have been “lifted” to apply to architecture. Metrics include component dependency, cyclicity and cyclomatic complexity, internal module complexity, module coupling and cohesion, levels of nesting, and compliance with the use of patterns, styles and (required) APIs.

In continuous development paradigms (such as DevOps), other metrics have evolved that focus not on the architecture directly but on the responsiveness of the process, such as metrics for lead time for changes, deployment frequency, mean time to restore service, and change failure rate — as indicative of the state of the architecture.

MATRIX OF TOPICS VS. REFERENCE MATERIAL

	Bass et al. [2*]	Budgen [6*]	Maier et al. [29*]	Rozanski et al. [38*]	Sommerville [40*]	Taylor et al. [41*]
1. Software Architecture Fundamentals	c1		Appendix C	c2		c1-3
<i>1.1. The Senses of "Architecture"</i>	c2	c6.1	c6			
<i>1.2. Stakeholders and Concerns</i>	c3-14			c8-9		c3
<i>1.3. Uses of Architecture</i>	c24			c30		
2. Software Architecture Description	c1.2,22			c12-13	c6	c6-7
<i>2.1. Architecture Views and Viewpoints</i>		c7-9	c8	c3	c6.2	
<i>2.2. Architecture Patterns, Styles and Reference Architectures</i>	c2.12	c6,15		c11	c6.3	c11
<i>2.3. Architecture Description Languages and Architecture Frameworks</i>	c22		c11	app		c6-7
<i>2.4. Architecture as Significant Decisions</i>				c8	c6.1	
3. Software Architecture Process			c12-13	c6-7		c4
<i>3.1. Architecture in Context</i>			c12-13			c2
<i>3.1.1. Relation of Architecture to Design</i>					c6	c2
<i>3.2. Architectural Design</i>	c19-23					
<i>3.2.1. Architecture Analysis</i>	c19					c8
<i>3.2.2. Architecture Synthesis</i>	c20					
<i>3.2.3. Architecture Evaluation</i>	c21			c14		
<i>3.3. Architecture Practices, Methods, and Tactics</i>	c3.4		c10	c9-14		

3.4. <i>Architecting in the Large</i>			c12,14		c19	
4. Software Architecture Evaluation	c21			c14		c8
4.1. <i>“Goodness” in Architecture</i>	c1.3,2	c17				
4.2. <i>Reasoning about Architectures</i>				c10		
4.3 <i>Architecture Reviews</i>	c21					
4.4 <i>Architecture Metrics</i>	c23					

FURTHER READINGS

Perry and Wolf, *Foundations for the study of software architecture* [34]

Perry and Wolf’s *Foundations* circulated informally for several years before its publication in 1992. It has indeed served as a foundation for the evolution of the discipline of software architecture, introducing a number of ideas that are fundamental to the field, including architecture as a discipline; distinguishing architecture and design; elements of software architectures; multiple views; architecture styles and types; and analogies with other fields.

Bass et al., *Software Architecture in Practice* [2*]

This book introduces concepts and recommended practices of software architecture, meaning how software is structured and how the software’s components interact. The book addresses several quality concerns in detail, including: availability, deployability, energy efficiency, modifiability, performance, testability and usability. The authors offer recommended practices focusing on architectural design, architecture description, architecture evaluation and managing architecture technical debt. They also emphasize the importance of the business context in which large software is designed. In doing so, they present software architecture in a real-world setting,

reflecting both the opportunities and constraints that organizations encounter.

Kruchten, *The 4+1 View Model of Architecture* [25].

This seminal paper organizes an approach to architecture description using five architecture viewpoints. The first four are used to produce the logical view, the development view, the process view, and the physical view. These are integrated through selected use cases or scenarios to illustrate the architecture. Hence, the model results in 4+1 views. The views are used to describe the software as envisioned by different stakeholders — such as end-users, developers, and project managers.

Rozanski and Woods, *Software Systems Architecture* [38*]

This is a handbook for the software systems architect. It develops key concepts of stakeholder, concern, architecture description, architecture viewpoint and architecture view, architecture patterns and styles, with examples. It provides an end-to-end architecting process. The authors provide a catalog of ready-to-use, practical viewpoints for the architect to employ that are applicable to a wide range of systems. The book is filled with guidance for applying these concepts and methods.

R.N. Taylor, N. Medvidović, E. Dashofy, *Software Architecture: Foundations, Theory, and Practice* [41*]

This is a comprehensive textbook on many aspects of software architecture, including key ideas; software architecture in the context of software engineering; the design process; architecture modeling, analysis and visualization; and chapters on several concerns including implementation, deployment, adaptation, non-functional properties, trust and security.

P. Clements et al. *Documenting Software Architecture: Views and Beyond*, 2nd edition [9].

This book provides best practices on capturing software architectures, using guidance and examples to express architectures that stakeholders can build, use, and maintain that system. The book introduces a 3-way categorization of views and therefore viewpoints: into module, component and connector, and allocation viewtypes, providing numerous examples of each.

Brown, *Software Architecture for Developers* [5]

Brown provides an overview of software architecture topics from the perspective of a developer. He discusses common architecture drivers including architecture principles, quality concerns, constraints and functional requirements. He has an in-depth discussion of the role of the architect in a development setting and requisite knowledge and skills for architects. He focuses on the practical issues of architecture in the delivery process and on managing risk. An appendix provides a case study.

Fairbanks, *Just Enough Software Architecture: A risk-driven approach* [16]

Fairbanks offers a risk-driven approach to architecting within the context of development: do just enough software architecture to mitigate the identified risks where those

risks could result from a small solution space, from extremely demanding quality requirements or from possible high-risk failures. The risk-driven approach is harmonious with low-ceremony and agile approaches. Architecting, as argued by Fairbanks, is not just for architects — but is relevant to all developers.

Erder, Pureur and Woods, *Continuous Architecture in Practice: Software Architecture in the Age of Agility and DevOps*. [15]

This book shows how “classical” thinking about software architecture has evolved in the present day in the contexts of agile, cloud-based and DevOps approaches to software development by providing practical guidance on a range of quality and cross-cutting concerns including security, resilience, scalability and integration of emerging technologies.

REFERENCES

- [1] M. Ali Babar, and I. Gorton, “Software Architecture Review: The State of the Practice”, *IEEE Computer*, July 2009.
- [2*] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th edition, 2021.
- [3] L. Bass, J. Ivers, M.H. Klein, and P. Merson, Reasoning Frameworks, CMU/SEI-2005-TR-007, 2005.
- [4*] F. Brooks, *The Design of Design*, Addison-Wesley, 2010.
- [5] S. Brown, *Software Architecture for Developers*, 2018, <http://leanpub.com/software-architecture-for-developers>.
- [6*] D. Budgen, *Software Design: Creating Solutions for Ill-Structured Problems*, 3rd Edition, CRC Press, 2021.
- [7] F. Buschmann, R. Meunier, H.

- Rohnert, P. Sommerlad, and M. Stal, *Pattern Oriented Software Architecture*, John Wiley & Sons, 1996.
- [8] H. Cervantes, R. Kazman, *Designing Software Architectures: A Practical Approach*, 2nd ed., Addison-Wesley, 2024.
- [9] P. Clements et al., *Documenting Software Architecture: Views and Beyond*, 2nd edition Addison-Wesley, 2011.
- [10] P. Clements, R. Kazman, M. Klein, *Evaluating Software Architectures*, Addison-Wesley, 2001.
- [11] M.E. Conway, “How Do Committees Invent?” *Datamation*, 14(4), 28-31, 1968.
- [12] E.W. Dijkstra, “On the role of scientific thought”, 1974, available at <https://www.cs.utexas.edu/users/EWD/transcriptions/EWD04xx/EWD447.html>.
- [13] T. Earl, *SOA Design Patterns*, Prentice-Hall, 2009
- [14] P. Eeles, and P. Cripps, *The Process of Software Architecting*, Addison Wesley, 2010.
- [15] M. Erder, P. Pureur and E. Woods, *Continuous Architecture in Practice: Software Architecture in the Age of Agility and DevOps*, Addison-Wesley, 2021.
- [16] G. Fairbanks, *Just Enough Software Architecture: A Risk-Driven Approach*, Marshall & Brainerd, 2010.
- [17] E. Fernandez-Buglioni, *Security Patterns in Practice: Designing Secure Architectures Using Software Patterns*, Wiley, 2013.
- [18] R.T. Fielding and R.N. Taylor, Principled design of the modern web architecture, *ACM Transactions on Internet Technology*, 2(2), 115–150, 2002.
- [19] M. Fowler, D. Rice, M. Foemmel, E. Hieatt, R. Mee and R. Stafford, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2003.
- [20] C. Hofmeister, P.B. Kruchten, R.L. Nord, H. Obbink, A. Ran, and P. America, “A general model of software architecture design derived from five industrial approaches”, *The Journal of Systems and Software*, 80, 106–126, 2007.
- [21] C. Hofmeister, R.L. Nord, and D. Soni, *Applied Software Architecture*, Addison-Wesley, 2000.
- [22] ISO/IEC/IEEE 24765:2017 Systems and Software Engineering — Vocabulary, 2nd ed. 2017.
- [23] ISO/IEC/IEEE 42010:2011, Systems and software engineering — Architecture description.
- [24] R. Kazman, S. Haziyevev, A. Yakuba, and D.A. Tamburri, Managing Energy Consumption as an Architectural Quality Attribute, *IEEE Software*, 35(5), 102–107, 2018
- [25] P.B. Kruchten, The “4+1” View Model of Architecture, *IEEE Software* 12(6), 1995.
- [26] P.B. Kruchten, R.L. Nord, and I. Ozkaya, *Managing Technical Debt: Reducing Friction in Software Development*. Addison-Wesley, 2019.
- [27] Z. Li, P. Liang and P. Avgeriou, Architecture viewpoints for documenting architectural technical debt. *Software Quality Assurance*, Elsevier, 2016.
- [28] Alan MacCormack, John Rusnak & Carliss Baldwin, Exploring the Duality between Product and Organizational

- Architectures: A Test of the ‘Mirroring’ Hypothesis. *Research Policy*, 41:1309–1324, 2012
- [29*] M.W. Maier and E. Rechtin, *The Art of Systems Architecting*, 3rd edition, CRC Press, 2021.
- [30] N. Medvidović, D.S. Rosenblum, D.F. Redmiles and J.E. Robbins, Modeling software architectures in the Unified Modeling Language, *ACM Transactions on Software Engineering and Methodology*, 11(1), 2–57, 2002
- [31] H. Obbink et al., *Report on Software Architecture Review and Assessment (SARA)*, version 1.0, available at <https://philippe.kruchten.com/architecture/SARAv1.pdf>, 2002.
- [32] D.L. Parnas, “On the criteria to be used in decomposing systems into modules”, *Communications of the ACM* 15(12), 1053–1058, 1972.
- [33] D.L. Parnas and D.M. Weiss, “Active Design Reviews: Principles and Practices”, Proceedings of 8th International Conference on Software Engineering, 215–222, 1985.
- [34] D. Perry, A. Wolf, Foundations for the study of software architecture, *ACM SIGSOFT Software Engineering Notes*, 17(4), 40–52, 1992
- [35] E. Poort, H. van Vliet, RCDA: Architecting as a Risk- and Cost Management Discipline, *Journal of Systems and Software*, <https://www.cs.vu.nl/~hans/publications/y2012/JSS-RCDA.pdf>, 2012
- [36] R. Prieto-Díaz and J.M. Neighbors, “Module Interconnection Languages”, *Journal of Systems and Software*, 6(4), 307–334, 1986.
- [37] C. Richardson, *Microservices Patterns*, Manning Publications, 2019
- [38*] N. Rozanski and E. Woods, *Software Systems Architecture: Working with Stakeholders Using Viewpoints and Perspectives*, 2nd edition, Addison-Wesley, 2011.
- [39] M. Shaw and D. Garlan, *Software Architecture: Perspectives on an Emerging Discipline*, Prentice Hall, 1996.
- [40*] I. Sommerville, *Software Engineering*, 10th edition, 2016.
- [41*] R.N. Taylor, N. Medvidović, E. Dashofy, *Software Architecture: Foundations, Theory, and Practice*, Wiley, 2009
- [42] R. Weinreich and G. Buchgeher, Towards supporting the software architecture life cycle, *The Journal of Systems and Software*, 85, 546–561, 2012.